

第六周周报

第 2 组 OS 课设周报-第 6 周-杜昊阳-富思涵-赫金科-康忆文-卢彦芸-陆昱衡-米梓润-吴安然

本周开始周报主要描述上周计划的执行情况及下周工作计划等内容

一、本周计划执行情况

1. 内存管理：杜昊阳，赫金科

- 功能整合 (apps/memmap/memmap.c:19-35, 87-93)
 - 在 memmap 可视化工具中显示当前内存分配算法
 - 添加 mem_algo_name()函数将算法 ID 转换为可读名称
 - 调用 api_getmemalgo()获取算法信息并显示在界面中
- 代码优化
 - 修复 kernel/console.c 文件编码问题，统一为 GB2312 编码

2. 中断：吴安然

完成定时器子系统系统测试与专项测试，验证压力场景下的正确性与稳定性，确认与任务切换、输入中断的协同无异常。

- 压力与正确性测试

运行 60 个并发定时器（随机超时 1~200 tick），观察是否出现错序、漏触发、死机现象；对 timer_cancel / timer_cancelall 做边界测试（取消已到期、未到期、已取消的定时器）。测试结果：所有定时器均正确触发，无错序或丢失。
- 与任务切换的协同验证

阅读 mtask.c 中 task_timer 与 inthandler20 的交互逻辑，确认改动未破坏时间片轮转。通过运行两个用户进程对比调度周期，结果与改动前一致。
- 与输入中断的协同验证

在定时器压力测试同时频繁敲击键盘，确认无竞态或响应延迟。内核中 IRQ0 与 IRQ1 均有关中断保护，未发现异常。
- 本周产出
 - a. 分条测试结果表格（用例 | 结果 | 备注）。

- b. 测试日志与截图存档。
- c. 与任务切换/输入中断协同的验证记录。

3. 进程调度：富思涵，卢彦芸，康忆文

- 设置监视窗口，定期刷新实验结果数据
- 设置内核态竞争条件

系统专门设计了两组对比任务来演示未保护情况下的“数据丢失”和保护情况下的“安全更新”：

- 无锁竞争 `task_race_unsafe`:

使用单纯的 `temp = g_race_unsafe_value; temp = temp + 1;` 操作，执行“读-改-写”非原子序列。在读取和写回之间穿插了 `sync_delay(18000)` 来放大竞争窗口。多个并发任务同时跑此函数时，极易发生 `Lost Update`（运行显示尝试总数 `attempts` 远大于实际成功增加的值 `value`）。

- 对内核态竞争条件设计解决方法

通过信号量解决竞争条件。

- 信号量实现：

- 实现了一个基础的 FIFO 机制信号量，数据结构定义了 `struct KSEMAPHORE` 提供信号量的基本结构，包含：可用资源计数 `value`、当前阻塞任务数量 `wait_count`、以及用于存放等待阻塞任务的 FIFO 队列 `waiters`。

- 操作原语：

- 初始化 `ksem_init`：初始化信号量可用资源值并清空等待队列。

- Wait 操作 `ksem_wait`：采用原子操作（`io_cli()` 和 `io_sti()` 开关中断保护）。有可用资源时 `value--`；无资源时，将当前任务加入到等待队列 `waiters` 并调用 `task_sleep` 让出 CPU 进入休眠。

- Signal 操作 `ksem_signal`：归还一个资源 `value++`。检测等待队列，如果有任务阻塞，则从 FIFO 队列头部移除任务并调用 `task_run` 唤醒它恢复运行。

- 加锁竞争 `task_race_safe`:

- 同样执行读、延时、写的逻辑，但是整个临界区首尾加上了 `ksem_wait(&g_sem_counter_lock)` 和 `ksem_signal(&g_sem_counter_lock)`。使用二值互斥信号量确保了并发写回的安全性，从而规避竞争条件，使总尝试数和值始终保持一致。

- 设计读写者场景

系统中通过信号量实现了典型的“读者优先”的读写模型，包含读者组 `task_rw_reader` 和写者组 `task_rw_writer`：

- 相关信号量：
 - `g_sem_rw_count`：专门用来保护共享变量 `g_rw_read_count` (当前活跃读者总数) 在递增/递减时不产生数据竞争。
 - `g_sem_rw_resource`：用于读写互斥锁，保障同一时刻只有多个读者或者一个写者。
- 读者机制 `task_rw_reader`：
 - 当一个读者要进行读操作时，先通过互斥获取修改读者数量的权限：如果它是首个读者 (`g_rw_read_count == 1`)，它会调用 `ksem_wait(&g_sem_rw_resource)` 抢占独占资源锁阻止写者介入，此后后续进入的读者可以直接读取；读完后，最后一个离开的读者 (`g_rw_read_count == 0`) 负责释放 `ksem_signal(&g_sem_rw_resource)` 将资源交还给写者。
- 写者机制 `task_rw_writer`：
 - 写者操作极度纯粹。通过请求独占的 `g_sem_rw_resource` 信号量来访问临界区。如果这时候有任何读者或其他写者在持有资源锁，写者会进行阻塞，直到独占并完成写操作再次释放资源。内部 `g_rw_violation_count` 变量检测读写是否同时发生以防范竞态逻辑错误违规。

4. 文件系统：米梓润，陆昱衡

- 解释了 LDT 在本工程中承担的安全边界，明确了其对用户程序访问权限的限制，防止用户程序直接读写内核地址空间。
- 梳理了工程中“用户态程序 (Ring3)”的运行模型，包括如何装载、如何切换到用户态、以及如何通过系统调用回到内核态。

二、下周工作计划

1. 内存管理：杜昊阳，赫金科

- 明确地址空间与权限策略
用户态与内核态页权限分离，先保留现有分段接口，分页负责实际访问控制。
- 建立分页基础设施
完成页目录与页表初始化，至少包含内核关键区域映射、应用运行所需映射、页对齐

与映射工具函数。

- 启用分页并保持分段兼容

在不破坏现有任务与应用加载逻辑的前提下开启分页，让系统先稳定跑在段页并存模式。

- 补齐异常处理与调试信息

完善缺页异常处理与日志，能定位权限错误、未映射访问、用户态越权访问。

- 回归验证

验证启动、任务切换、系统调用、应用运行与退出流程。

2. 中断：吴安然

- 说明「更多定时器」相对原实现的差异（池规模扩展、测试命令、调试输出）。
- 总结 TIMERCTL 链表法的时空特点（插入 $O(n)$ 、每 tick 摊销 $O(1)$ 等）。
- 与组内衔接，注明实验基于的提交版本，避免验收时版本不一致。

3. 进程调度：富思涵，卢彦芸，康忆文

- 设计并实现从用户态启动内核态竞争条件、解决方案、同步场景（练习系统调用）
- 设计并实现用户态程序通过系统调用进行同步产生竞争条件，提供解决方案并实现一个同步场景（比如生产者消费者）

4. 文件系统：米梓润，陆昱衡

- 设计并实现一个可演示方案：在用户程序中选取不同类型的变量（全局/局部/静态/堆），展示它们从逻辑地址到物理地址的转换过程，并进行验证。
- 完善部分文件操作指令